COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010- Data structure and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	Aug 18, 2026	**DATE SUBMITTED:**	Aug 25, 2026
NAME:	Mauro, Ryan Lewis B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 6.1 [SEARCHING TREE]

1. PROCEDURE OUTPUT

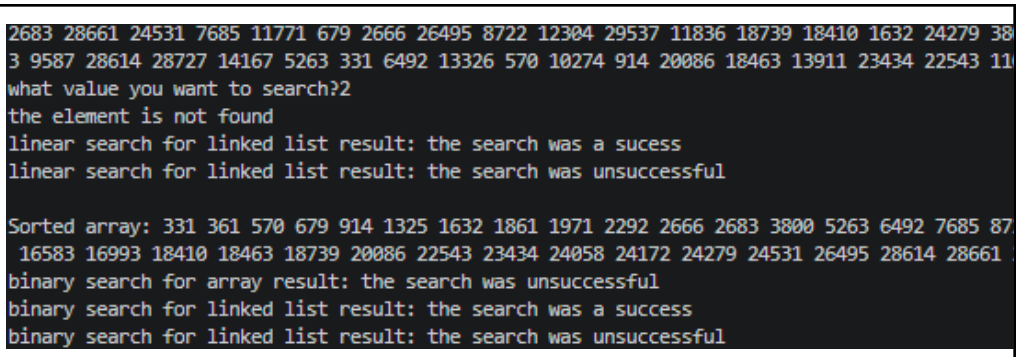
Screenshot	
Observation	The program successfully generated **50 random integer values** and displayed them in the console. The user entered **2** as the value to search. Since **2 was not present** in the generated dataset, the program reported that the element was not found. The array was then sorted successfully for the binary search process.

Table 6-1. Data Generated and Observations.

Screenshot	

Output	
Observation	The linear search checks each array element one by one from the beginning until the target value is found or the end of the array is reached. It can search an unsorted array.

Table 6-2a. Linear Search for Arrays

Screenshot	**.h file:**
	`template <typename T>`
	`void linearSearchLL(Node<T>* node, T dataFind){`
	
	`//repeat the loop until the current node is not null`
	`while(node != nullptr){`
	
	`//if item = current node's data then display search is successful`
	`if(dataFind == node->data){`
	`std::cout<<"the search was a sucess\n";`
	`return;`
	`}`
	
	`//if not, move to the next node`
	`node = node->next;`
	`}`
	
	`//the searching reached nullptr -> search is unsuccessful`
	`std::cout<<"the search was unsuccessful \n";`
	`return;`
	`}`
	Main.cpp file:
	`std::cout <<"linear search for linked list result: ";`
	`linearSearchLL(name1, '7');`
	
	`std::cout <<"linear search for linked list result: ";`
	`linearSearchLL(name1, 'X');`

Output	`linear search for linked list result: the search was a sucess` `linear search for linked list result: the search was unsuccessful`
Observation	The linear search successfully found '7' in the linked list. When searching for 'X', the function checked each node until it reached the end of the list and determined that the value was not found.

Table 6-2b. Linear Search for Linked List

Screenshot	**.h file:**
	`template <typename T>`
	`int getMiddle(T data[], int first, int last){`
	
	`    return first + (last - first) / 2;`
	`}`
	
	`template <typename T>`
	`void binarySearchArr(T data[], int arrSize, T findData){`
	
	`    int first = 0;`
	`    int last = arrSize - 1;`
	
	`    while(first <= last){`
	
	`        //get the middle index`
	`        int middle = getMiddle(data, first, last);`
	
	`        //if middle contains the data`
	`        if(data[middle] == findData){`
	`            std::cout << "the search was a success\n";`
	`            return;`
	`        }`
	
	`        //if findData is greater than middle data`
	`        if(data[middle] < findData){`
	`            first = middle + 1;`
	`        }`
	
	`        //if findData is smaller than middle data`
	`        else{`
	`            last = middle - 1;`
	`        }`
	`    }`
	
	`    std::cout << "the search was unsuccessful\n";`
	`    return;`
	`}`
	Main.cpp:
	`std::cout << "binary search for array result: ";`
	`binarySearchArr(dataset, max_size, toFind);`

Output	`binary search for array result: the search was unsuccessful`
Observation	The array was sorted before performing the binary search. The algorithm repeatedly checks the middle element and eliminates half of the remaining search area. This makes binary search more efficient than linear search for a sorted array.

Table 6-3a. Binary Search for Arrays

Screenshot	**.h file:**
	`template <typename T>`
	`void binarySearchLL(Node<T> *start, T findData){`
	
	`    Node<T>* last = nullptr;`
	
	`    while(start != last){`
	
	`        //get the middle node`
	`        Node<T>* middle = getMiddle(start, last);`
	
	`        //if middle contains the data`
	`        if(middle->data == findData){`
	`            std::cout << "the search was a success\n";`
	`            return;`
	`        }`
	
	`        //if findData is greater than middle data`
	`        if(middle->data < findData){`
	`            start = middle->next;`
	`        }`
	
	`        //if findData is smaller than middle data`
	`        else{`
	`            last = middle;`
	`        }`
	`    }`
	
	`    std::cout << "the search was unsuccessful\n";`
	`    return;`
	`}`
	Main.cpp:
	`std::cout << "binary search for linked list result: ";`
	`binarySearchLL(num1, 40);`
	
	`std::cout << "binary search for linked list result: ";`
	`binarySearchLL(num1, 100);`

Output	`binary search for linked list result: the search was a success` `binary search for linked list result: the search was unsuccessful`
Observation	The binary search successfully found 40 in the sorted linked list. When searching for 100, the algorithm checked the appropriate portions of the list and determined that the value was not present. The linked list must be sorted for binary search to work correctly.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

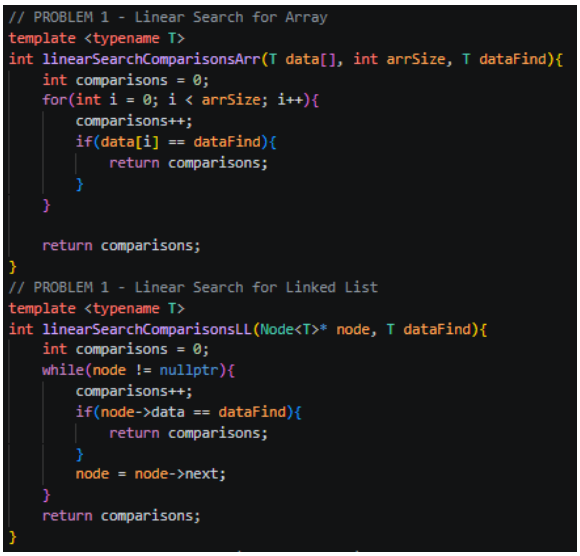
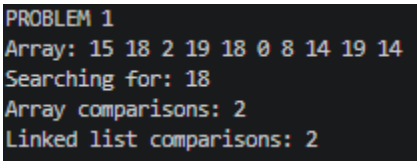
Screenshot	
Output	
Observation	The sequential search requires 2 comparisons for both the array and linked list because the first occurrence of 18 is the second element in the list.

Table 6-4. Problem 1

```
// PROBLEM 1 - Linear Search for Array
template <typename T>
int linearSearchComparisonsArr(T data[], int arrSize, T dataFind){
    int comparisons = 0;
    for(int i = 0; i < arrSize; i++){
        comparisons++;
        if(data[i] == dataFind){
            return comparisons;
        }
    }

    return comparisons;
}
// PROBLEM 1 - Linear Search for Linked List
template <typename T>
int linearSearchComparisonsLL(Node<T>* node, T dataFind){
    int comparisons = 0;
    while(node != nullptr){
        comparisons++;
        if(node->data == dataFind){
            return comparisons;
        }
        node = node->next;
    }
    return comparisons;
}
```

```
PROBLEM 1
Array: 15 18 2 19 18 0 8 14 19 14
Searching for: 18
Array comparisons: 2
Linked list comparisons: 2
```

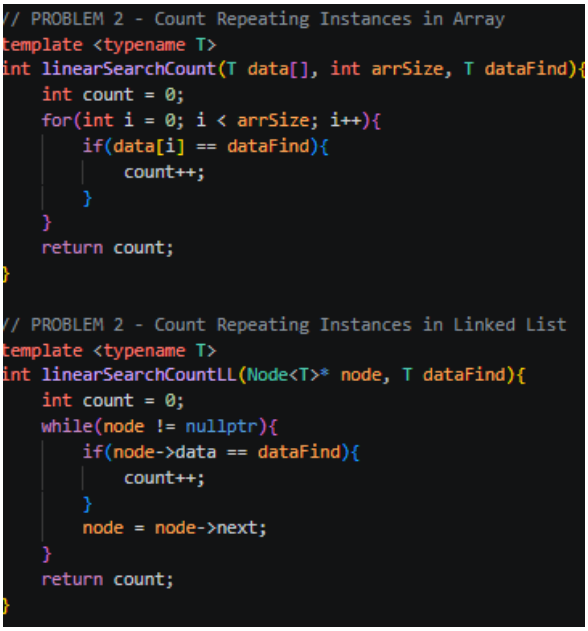
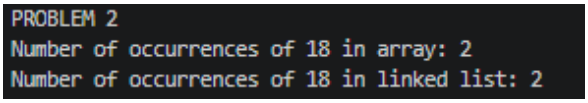
Screenshot	
Output	
Observation	The modified sequential search continued checking all elements instead of stopping after finding the first occurrence. The key 18 appears twice in the given list, at the second and fifth positions. Therefore, the algorithm returned a count of 2 occurrences for both the array and linked-list implementations. This demonstrates that the modified algorithm can identify repeated values in the data.

Table 6-5. Problem 2

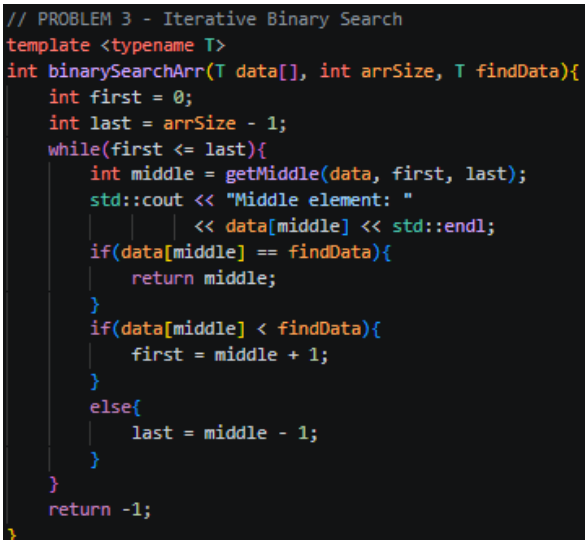
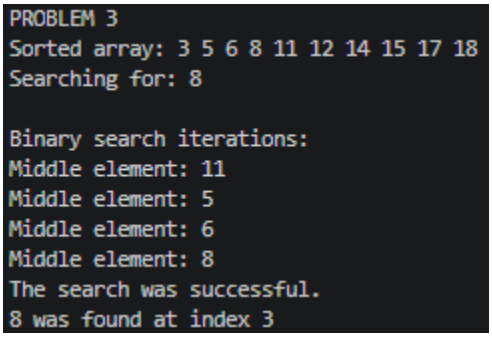
Screenshot	
Output	
Observation	The binary search was performed on the sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18]. The algorithm first checked the middle value 11. Since 8 is smaller than 11, the search continued in the left half. It then checked 5, followed by 6, and finally found 8. The key was found after 4 comparisons. This shows how binary search reduces the search area after every comparison.

Table 6-6. Problem 3

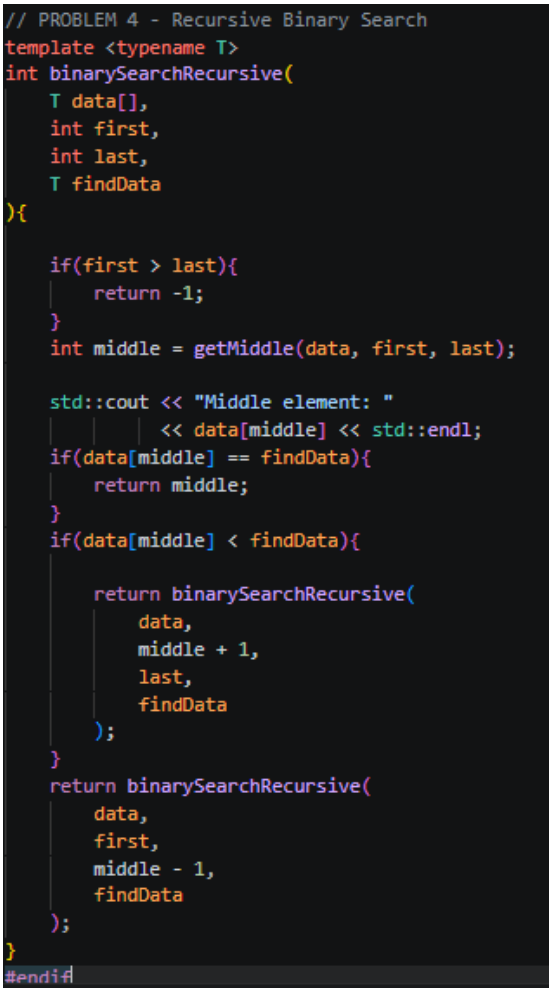
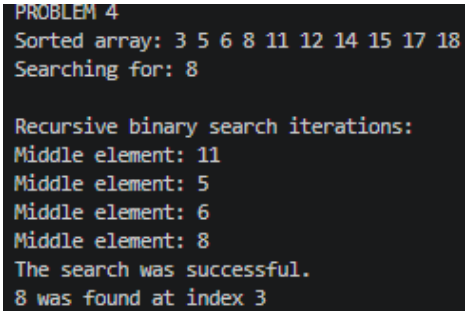
Screenshot	
Output	
Observation	The recursive binary search follows the same sequence as the iterative binary search: 11, 5, 6, and 8. The difference is that the recursive version calls itself with a smaller search range after each comparison until the value is found.

Table 6-7. Problem 4

Conclusion/learnings:
In this activity, I learned how to implement and apply different searching techniques in C++ using arrays and linked lists. I gained a better understanding of how sequential search works by checking elements one at a time until the target value is found, and how binary search improves efficiency by repeatedly dividing a sorted list into smaller sections. I also learned how to modify a sequential search algorithm to count multiple occurrences of a value and how to implement binary search using both iterative and recursive approaches. Through the exercises, I observed that sequential search is suitable for unsorted data, while binary search is much more efficient when the data is sorted. The activity strengthened my understanding of searching algorithms, recursion, and data structures, and highlighted the importance of selecting the most appropriate algorithm based on the characteristics of the data and the problem being solved.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.